Informatique

IUT Nord Franche-Comté

Implementation of Algorithmic Strategies for Nine Men's Morris

SAE Project Report

Authors :

Adam BAJIC (A1)

Lukas BENDIAF (A1)

Valentin BEURET (A1)

Lucas LAPINOT (A1)

Supervisor : Mr. Domas

22/05/2026

Table of Content

Chapter 1 : Project Context and Objectives

1.1 Presentation of Nine Men's Morris

The game of Nine Men's Morris, also known as the mill game, is a very ancient two-player strategic board game, already played by the Egyptians and the Romans. In the 14th-century, it became particularly popular in France.

The game is played on a board composed of twenty-four interconnected positions. Each player owns nine pawns and must place and move them in order to create "mills", which are alignments of three pawns. Forming a mill allows the player to remove one opponent pawn from the board.

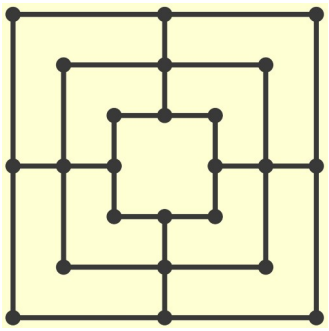

Figure 1: An example of the game board

The game is divided into two main phase :

- The placement phase, during which players alternately place their pawns on empty positions.
- The movement phase, during which players move their pawns between adjacent positions.

A player loses when they have fewer than three pawns remaining or when they are no longer able to perform any valid move.[1]

Due to its relatively simple rules but highly strategic gameplay, Nine Men's Morris is particularly suitable for experimenting with computer algorithms based on decision making trees and game simulations.

1 Tie was also added to the game to prevent infinite repetition of moves when two algorithms play against each other.

1.2 Objectives of the Project

One of the objective of the project was to design and implement at least 2 strategies capable of playing Nine Men's Morris autonomously.

Three different approches were developped

- A MiniMax-based strategy
- An optimized Alpha-Beta strategy
- A Monte Carlo simulation strategy

Alpha-Beta use the same principle as the MiniMax startegy but optimized so we decided to separate the two and also analyse both of them.

1.3 Challenges for those Algorithms

Developing an efficient algorithm for Nine Men's Morris introduces a lot of algorithmic challenges.

First, the number of possible game states grows rapidly as the game progresses. Exploring every possible move becomes computationally expensive.

Second, evaluating the quality of a position is difficult because a good move is not always immediately beneficial. The algorithm must therefore rely on heuristic evaluation[2] functions capable of estimating the strategic value of a board configuration.

And finally, an important challenge concerns repetitive behaviors. Without additional constraints, artificial intelligences may repeatedly perform inverse moves or continuously reform the same mill, leading to unrealistic or infinite games.

2 A heuristic evaluation function is a scoring formula that estimates how favourable a board position is for a given player, without searching all the way to a terminal state (win/loss). Since exhaustive search is computationally infeasible beyond a few moves, the algorithm evaluates intermediate positions using hand-crafted criteria such as pawn count, number of mills formed, threats, and mobility. The quality of the heuristic directly determines the quality of the AI's decisions.

Chapter 2 : Global architecture of the Algorithms

2.1 General Decision Process

The artificial intelligence system is centralized within a single decision-making class responsible for selecting the most appropriate move depending on the chosen strategy.

At each turn, the AI follows a general pipeline:

1) The current state of the board is converted into a simplified internal representation.
2) All legal moves are generated according to the game phase (placement, movement, or capture).
3) Each possible move is evaluated using a selected strategy.
4) The move with the best evaluation score or simulation outcome is selected and returned.

This architecture ensures the clear separation between the game state representation, the move generation and the decision-making algorithms.

2.2 Board Representation

In order to optimize performance during simulations, the board is a lightweight array-based representation : each index of this array corresponds to one position on the board.

This design eliminates all boardifier event propagation overhead during tree search, makes deep copying trivial via snap.clone(), and allows large numbers of positions to be evaluated per second without garbage-collection pressure.

2.3 Move Generation

Move generation is handled differently depending on the current phase of the game.

Three types of actions are considered:

⇒ Placement phase: the AI selects any empty position on the board.

⇒ Movement phase: the AI moves a pawn to an adjacent empty position.

⇒ Capture phase: when a mill is formed, the AI selects an opponent pawn to remove.

These actions are generated using helper functions that operate directly on the snapshot representation.[3]

For instance, all the empty positions are retrieved for placement then the adjacency lists define valid movements and the capture rules respect the constraint that non-mill pawns must be prioritized unless all opponent pawns are in mills.

2.4 General Constant as a Base

Constant	Value	Meaning
MINIMAX_DEPTH	4	Search depth for MiniMax
ALPHABETA_DEPTH	6	Search depth for Alpha-Beta
MCTS_SIMULATIONS	80	Random play-outs per candidate move

3 A snapshot is a lightweight, self-contained copy of the board state stored as a plain integer array (int[24]). Rather than manipulating the full game objects during tree search, the AI works exclusively on these snapshots. Copying a snapshot is a single clone() call on a 24-element array, making recursive tree exploration fast and side-effect free.

Chapter 3 : MiniMax Strategy

3.1 Principle of the MiniMax Algorithm

The MiniMax strategy is a classical adversarial search algorithm used in two-player zero-sum games. It is based on the assumption that both players play optimally: the AI attempts to maximize its advantage while the opponent attempts to minimize it.

The algorithm explores a game tree where the MAX nodes correspond to the AI's turns and the MIN nodes correspond to the opponent's turns.

At each level of the tree, all possible legal moves are generated and recursively evaluated until a given depth is reached or a terminal state is encountered.

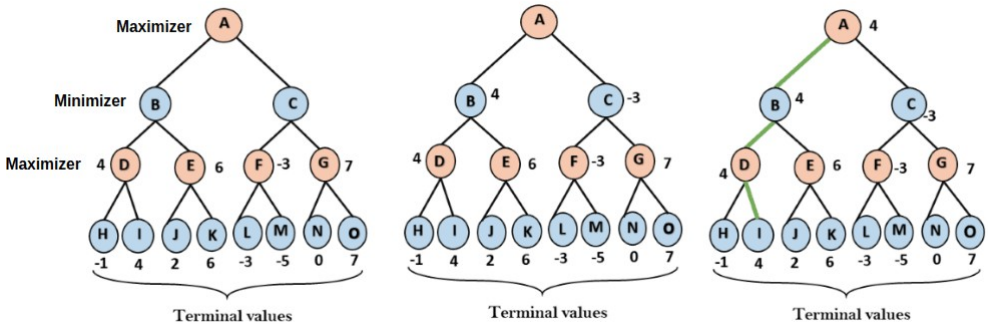

Figure 2: Generic exemple of a MiniMax tree.

The final decision is obtained by selecting the move that leads to the best evaluated outcome from the perspective of the AI.

With MINIMAX_DEPTH = 4, the tree reaches up to four half-moves (plies) ahead. Each ply may branch into up to approximately 8 placement options in Phase 1 or 4–6 movement options in Phase 2, giving a worst-case tree of roughly 4,096 nodes in Phase 1 or 1,296 nodes in Phase 2 before pruning.

3.2 Implementation Details

The entry point getDecisionMinimax() handles the three decision contexts (capture, placement, movement) and then calls the recursive minimax() method on int[24] snapshots. Mill detection is integrated inline: if a placement or move forms a mill via formsMillSnap(), the algorithm branches into all possible captures before continuing the search, correctly modelling the compulsory capture rule.

The key main implementaion choices are: snapCopy(snap) performs an O(24) array clone at negligible cost; allMovesSnap(), allPlacementsSnap(), and allCapturesSnap() are pure array scans with no iterator overhead; terminal detection in isTerminalSnap() checks pawn count below 3 or no legal moves in Phase 2.

3.3 Heurisitc Evaluation Function

The shared evaluateSnap() function scores a position from the AI's perspective. A positive score favours the AI; a negative score favours the opponent. The weighting scheme, in descending priority, is shown below.

Criterion	Weight	Rationale
Terminal WIN / LOSS	± 100,000	Immediate game-over signal
Mills formed (AI − Opp)	× 5,000	Dominant winning condition
Mill threats (2-in-a-row, open slot)	× 2,000	Prevent / build mills
Pawn count (AI − Opp)	× 1,000	Material advantage
Dead-position penalty	−800	Discourage stalemate loops
Mobility (legal moves, AI − Opp)	× 50	Freedom of action

3.4 Anti-Loop Mechanism

Two penalties are applied at the root level to prevent oscillating behaviour. The ping-pong penalty (−500 pts) fires when the candidate move is the exact reverse of the AI's last recorded move (e.g. A1→B1 followed by B1→A1). The mill-reformation penalty (−100,000 pts) fires when a move would re-form the exact same mill the AI last formed, tracked via stageModel.isSameMillAsLast(), forcing the AI to vary its play before re-using the same mill.

3.5 Computational performance

Metric	Value	Source
Search depth	4 plies	MINIMAX_DEPTH constant
Average decision time	~5,897 µs (~6 ms)	Benchmark (2,000 games)
Min / Max decision time	0 ms / 53 ms	Benchmark vs Alpha-Beta
Total CPU time (2,000 games)	~867 s	Benchmark total
Memory per snapshot (int[24])	96 bytes	24 × 4 bytes
Code size (MiniMax methods)	~120 lines	MerelleDecider.java
Average mills formed per game	4.72	Benchmark global ranking

Chapter 4 : Alpha-Beta Strategy

4.1 Principle

Alpha-Beta pruning[4] is a direct optimisation of MiniMax that maintains two window bounds during tree traversal. Alpha (α) is the best score the maximising player is already guaranteed on the current path; Beta (β) is the best score the minimising player is already guaranteed. Whenever $\alpha \geq \beta$ at any node, the remaining subtree cannot influence the final decision and is pruned without being explored.

In the best case with perfect move ordering, Alpha-Beta reduces the effective branching factor from b to $\sqrt{b}$, theoretically allowing twice the search depth for the same computation. This is why ALPHABETA_DEPTH = 6, two plies deeper than MiniMax, without exceeding the time budget per move. Two extra plies correspond to one complete exchange of moves, allowing the AI to anticipate traps and counter-strategies invisible to the depth-4 MiniMax player.

4.2 Pruning Mechanism

The recursive alphabeta() function is structurally identical to minimax() with the addition of α/β parameters and an immediate pruning check after each child is evaluated.

The cut fires inside the move loop before the next sibling is generated. Mill-capture branching is fully preserved: when a placement or move forms a mill, the algorithm branches into all possible captures before applying any cut, ensuring the compulsory capture rule creates no blind spots. The same anti-ping-pong (−500 pts) and anti-mill-reformation (−100,000 pts) penalties present in MiniMax are also applied at the root level in Alpha-Beta.

4.3 Computational Performance

Despite searching two plies deeper, the benchmark shows that Alpha-Beta's average decision time is roughly 190,297 µs — substantially higher than MiniMax's 5,897 µs. This surprising result is explained by the specific nature of Nine Men's Morris: the branching factor is low (typically 4–8 moves), so the absolute gain from pruning is modest in terms of nodes cut, while the two additional plies multiply the work significantly.

4 In the context of game tree search algorithms such as Alpha-Beta, pruning refers to the process of cutting off branches of the search tree that cannot possibly influence the final decision. When the algorithm determines that a given branch will never be chosen by either player — because a better option has already been found elsewhere — it stops exploring that branch entirely. This reduces the number of positions evaluated without affecting the correctness of the result.

The high maximum observed (544,266 ms in one extreme case) suggests that in rare complex positions — typically during Phase 1 with many mill opportunities branching into captures — the tree can expand far beyond the average.

Metric	Value	Source
Search depth	6 plies	ALPHABETA_DEPTH constant
Average decision time	~190,297 µs (~190 ms)	Benchmark (2,000 games)
Min / Max decision time	7 ms / 544,266 ms	Benchmark vs MiniMax
Total CPU time (2,000 games)	~28,475 s	Benchmark total
Memory per snapshot (int[24])	96 bytes	24 × 4 bytes
Code size (Alpha-Beta methods)	~150 lines	MerelleDecider.java
Average mills formed per game	3.40	Benchmark global ranking
Invalid moves generated	15 total / 2,000 games	Benchmark warning log

Chapter 5 : Monte Carlo Strategy

5.1 Principle

The Monte Carlo strategy belongs to a family of simulation-based algorithms that require no explicit evaluation function. Instead of searching a tree analytically, the algorithm evaluates each candidate move by running MCTS_SIMULATIONS = 80 random play-outs (rollouts) from the resulting position to a terminal state or turn limit. The move with the highest win count is selected.

The algorithm's theoretical advantage is that, given enough simulations, its win-rate estimate converges to the true probability of winning from a given position, independently of domain knowledge. It is particularly well-suited to games where designing a reliable heuristic is difficult or where the branching factor makes exhaustive tree search impractical.

5.2 Random Simulation Engine

Each rollout is handled by simulateRandomGameSnap(), which plays a full random game from a given snapshot up to MAX_TURNS = 80 half-moves. The simulation engine is phase-aware: it correctly handles both Phase 1 (placement) and Phase 2 (movement), including the automatic phase transition when all 18 pawns have been placed. When a random move forms a mill, a random opponent pawn is immediately captured, correctly modelling the rules.

The simulation also includes an anti-ping-pong filter that tracks each player's last move within the rollout and filters out the immediate reverse, preventing trivial infinite loops. If the turn limit is reached without a terminal state, the position is scored by evaluateSnap() and the result is determined by the sign of the score. At the root level, ping-pong and mill-reformation filters are applied before any simulation is launched, so known bad moves are never evaluated.

5.3 Computational Performance

Monte Carlo's average decision time is approximately 42,284 µs, which places it between MiniMax and Alpha-Beta in raw per-move cost. However, its total benchmark runtime (2,000 games) was approximately 4,058 s, far less than Alpha-Beta's 28,475 s, because games involving Monte Carlo terminate much faster (average 94–106 half-turns vs. 192 half-turns for MiniMax vs. Alpha-Beta).

The algorithm's memory footprint remains bounded since no persistent tree structure is maintained across decisions.

Metric	Value	Source
Simulations per candidate move	80	MCTS_SIMULATIONS constant
Average decision time	~42,284 µs (~42 ms)	Benchmark (2,000 games)
Min / Max decision time	5 ms / 807 ms	Benchmark vs Alpha-Beta
Total CPU time (2,000 games)	~4,058 s	Benchmark total
Max turn limit per rollout	80 half-turns	MAX_TURNS constant
Memory per snapshot (int[24])	96 bytes	24 × 4 bytes
Code size (Monte Carlo + engine)	~180 lines	MerelleDecider.java
Average mills formed per game	6.96	Benchmark global ranking

Chapter 6 : Experimental Results

6.1 Test Protocol

All matchups were run using an automated benchmark harness (MerelleSimulator) with aiDifficultyPerPlayer set independently for each player. The benchmark ran 1,000 games per matchup across 3 combinations (MiniMax vs. Alpha-Beta, MiniMax vs. Monte Carlo, Alpha-Beta vs. Monte Carlo), for a total of 3,000 games. The first player was fixed (no colour alternation) for all 1,000 games of each matchup, so first-player advantage is fully reflected in the results. A game was declared a draw (Timeout) if it exceeded 70 consecutive turns without progress. All measurements were taken on a single-threaded JVM (JDK 25.0.1 HotSpot) on an Intel Core i7, with a total benchmark duration of 33,427 s (~9.3 hours).

6.2 Detailed Results by Matchup

Matchup 1 – MiniMax vs Alpha-Beta

Metric	MiniMax (P1)	Alpha-Beta (P2)
Wins	487 (48.7%)	120 (12.0%)
Draws (Timeout ≥70 turns)	393 (39.3%)	—
Avg. game length (half-turns)	192.1	—
Avg. decision time per move	6,555 µs	277,759 µs
Total CPU time	639 s	26,280 s
Mills formed per game	3.65	1.55
End by < 3 pawns	427 (42.7%)	—
End by blocked player	180 (18.0%)	—
Invalid moves generated	0	4

Matchup 2 – MonteCarlo vs. MiniMax

Metric	MiniMax (P1)	Monte Carlo (P2)
Wins	813 (81.3%)	182 (18.2%)
Draws (Timeout ≥70 turns)	5 (0.5%)	—
Avg. game length (half-turns)	94.4	—
Avg. decision time per move	4,606 µs	41,910 µs
Total CPU time	229 s	1,876 s
Mills formed per game	5.79	1.72
End by < 3 pawns	864 (86.4%)	—
End by blocked player	131 (13.1%)	—

Matchup 3 – AlphaBeta vs. MonteCarlo

Metric	Alpha-Beta (P1)	Monte Carlo (P2)
Wins	731 (73.1%)	259 (25.9%)
Draws (Timeout ≥70 turns)	10 (1.0%)	—
Avg. game length (half-turns)	106.2	—
Avg. decision time per move	39,895 µs	42,611 µs
Total CPU time	2,195 s	2,183 s
Mills formed per game	5.25	2.19
End by < 3 pawns	842 (84.2%)	—
End by blocked player	148 (14.8%)	—
Invalid moves generated	11	0

6.3 Global Ranking

Rank	Strategy	Wins / 2000	Win Rate	Avg. µs/move	Mills/game
1st	MiniMax	1,300	65.0%	5,897 µs	4.72
2nd	Alpha-Beta	851	42.6%	190,297 µs	3.40
3rd	Monte Carlo	441	22.1%	42,284 µs	1.96

Winner \ Opponent	MiniMax	Alpha-Beta	Monte Carlo
MiniMax	—	487 (80.2%)	813 (81.7%)
Alpha-Beta	120 (19.8%)	—	731 (73.8%)
Monte Carlo	182 (18.3%)	259 (26.2%)	—

6.4 Analysis

Across all three matchups, MiniMax emerges as the strongest strategy overall (65.0% global win rate), which is counter-intuitive given that Alpha-Beta searches two plies deeper using the same evaluation function.

The explanation lies in the game-length data: MiniMax vs. Alpha-Beta produces the longest games of all matchups (avg. 192.1 half-turns) and the highest draw rate (39.3%), indicating that both algorithms detect the same threats and produce symmetric, mirroring responses that neutralise each other — extra depth does not help Alpha-Beta convert positional advantages into victories.

Compounding this, Alpha-Beta's average decision time (277,759 µs) is roughly 42× slower than MiniMax's (6,555 µs), and its depth-6 tree can explode to an observed maximum of 544,266 ms in complex Phase 1 positions with many mill-forming branches, even with pruning.

Against Monte Carlo, both tree-search strategies dominate decisively — MiniMax wins 81.3% and Alpha-Beta wins 73.1% — because 80 random rollouts per candidate move produce a noisy, unreliable win-rate estimate in Nine Men's Morris, where a random game from mid-position diverges sharply from a well-played one; MiniMax's heuristic, while imperfect, provides a far more stable signal.

The Alpha-Beta vs. Monte Carlo matchup is particularly telling since both strategies spend nearly identical time per move (~40 ms), yet Alpha-Beta wins nearly three times as often, confirming that structured search outperforms flat simulation at equal computational budget.

In terms of speed, MiniMax is by far the fastest (~5,897 µs/move), Monte Carlo sits in the middle (~42,284 µs/move), and Alpha-Beta is paradoxically the slowest (~190,297 µs/move) — its depth gains less than it costs at these game parameters, making MiniMax the best performance-to-speed ratio for Nine Men's Morris.

Chapter 7 : Input Files and Demonstration

7.1 Error_Menu.txt

This file demonstrates the menu errors caused by invalid numeric input.

7.2 Error_pawn_placement.txt

This file demonstrates the numerous errors that can occur during pawn placement, such as selecting an invalid position (outside the grid) or attempting to place a pawn on an already occupied position.

7.3 Error_pawn_mouvement.txt

This file demonstrates the numerous errors that can occur during pawn movement, such as selecting an invalid source position, choosing an invalid destination, moving outside the grid, attempting to move to a non-adjacent position, or trying to move an opponent's pawn.

7.4 Error_mill_formation.txt

This file demonstrates the numerous errors that can occur during mill formation, such as attempting to capture one's own pawn, trying to capture a non-existent pawn, selecting an invalid position, attempting to reform the same mill repeatedly, and trying to capture a protected pawn belonging to an opponent's mill.

7.5 Game_until_end.txt

This file showcases a normal game to demonstrate how the gameplay unfolds.

Chapter 8 : Conclusion

This project implemented and benchmarked three AI strategies for Nine Men's Morris: MiniMax (depth 4), Alpha-Beta pruning (depth 6), and flat Monte Carlo (80 rollouts). The results across 3,000 automated games produce a clear and perhaps surprising ranking:

Rank	Strategy	Win Rate	µs/move	Mills/game	Draws
1st	MiniMax (d=4)	65.0%	5,897 µs	4.72	39.3%*
2nd	Alpha-Beta (d=6)	42.6%	190,297 µs	3.40	39.3%*
3rd	Monte Carlo (n=80)	22.1%	42,284 µs	1.96	~0.7%

MiniMax emerges as the strongest strategy in this implementation, which is unexpected given that Alpha-Beta searches deeper with the same evaluation function. The explanation is a combination of Alpha-Beta's much higher per-move computation cost in complex positions (up to 544 seconds in one extreme case), a high draw rate when facing MiniMax suggesting both algorithms neutralise each other, and a small number of invalid moves generated by the deeper search near phase transitions.

Monte Carlo, despite being theoretically sound, is the weakest at 80 simulations per move. The high variance of random play-outs in Nine Men's Morris means that 80 rollouts is insufficient to produce a reliable win-rate signal, and the absence of any strategic guidance (no heuristic weighting of move ordering) makes it easy prey for both tree-search strategies.

Possible improvements for future work include: increasing MCTS_SIMULATIONS to 500–1,000 for stronger Monte Carlo play; adding move ordering (e.g. mill-forming moves first) to Alpha-Beta to improve pruning efficiency and reduce worst-case explosion; implementing iterative deepening for Alpha-Beta to impose a strict time budget per move rather than a fixed depth; and extending the anti-loop mechanisms to handle longer repetition cycles beyond the immediate ping-pong.